

ASSESSMENT TOOL 2

Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

I LOSHINI.L 192571015__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

LOSHINI,L

Case study

QUESTIONS: Any 4

Total Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

(10 Marks)

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

(10 Marks)

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

(10 Marks)

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.
(10 Marks)

5. Smart Healthcare Management System

A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

Question:

Analyze the system and explain how **object-oriented principles (encapsulation, inheritance, and polymorphism)** can be applied. Also, recommend a suitable **SDLC model** for developing this healthcare system and justify your choice.
(10 Marks)

6. Smart Library Management System

A university library intends to automate its operations, allowing students and faculty members to search books, borrow and return books, reserve titles online, and pay overdue fines. Librarians manage book inventories, memberships, and circulation records, while administrators generate reports and maintain the system.

Question:

Design a structured solution using **object-oriented concepts** for this system. Identify the major classes, attributes, methods, and relationships, and recommend an appropriate **SDLC model** for efficient development with suitable justification.
(10 Marks)

Question 1

Online Hotel Reservation System

1. System Understanding, Actors, and Use Cases

The platform lets customers search hotels, book rooms, pay online, and post reviews, while hotel managers update room availability and pricing and administrators manage users and transactions.

Actor	Key Use Cases
Customer	Search Hotels, View Room Availability, Book Room, Make Payment, Write Review, Cancel Booking
Hotel Manager	Update Room Availability, Update Pricing, View Bookings
Administrator	Manage Users, Manage Transactions, Generate Reports
Payment Gateway (external)	Process Payment, Send Payment Confirmation

2. Application of Object-Oriented Principles

- **Encapsulation:** The Room class hides its availability count and internal validation behind reserve() and release() methods, and Booking hides confirmation logic behind confirmBooking()/cancelBooking(), so no other part of the system can corrupt booking state directly.
- **Inheritance:** An abstract User class defines shared attributes (userId, name, email) and login()/logout() behaviour; Customer, HotelManager, and Administrator each extend User and add role-specific behaviour such as bookRoom() or updatePricing(), avoiding duplicated account-management code.
- **Polymorphism:** A single processPayment() call on a Payment reference executes different logic depending on whether the actual object is a CreditCardPayment, UPIPayment, or WalletPayment, so new payment methods can be added without changing the checkout workflow.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
User (abstract)	userId, name, email, password	login(), logout(), updateProfile()
Customer	loyaltyPoints, bookingHistory	searchHotels(), bookRoom(), writeReview()
HotelManager	hotelId	updateAvailability(), updatePricing()
Administrator	accessLevel	manageUsers(), viewTransactions()
Hotel	hotelId, name, location, rating	listRooms(), getAvailableRooms()
Room	roomNo, type, price, status	reserve(), release()
Booking	bookingId, checkIn, checkOut, status	confirmBooking(), cancelBooking()
Payment (abstract)	paymentId, amount, date, status	processPayment(), refund()
Review	reviewId, rating, comment	postReview()

4. Class Relationships

- Generalization: User is the superclass of Customer, HotelManager, and Administrator; Payment is the superclass of CreditCardPayment and UPIPayment.
- Composition: Hotel is composed of Room objects — a Room cannot meaningfully exist outside its owning Hotel.
- Association: Customer makes a Booking (1-to-many); Booking has one Payment (1-to-1); Customer writes many Review objects (1-to-many).

5. SDLC Model Recommendation: Agile (Scrum)

- The system serves three distinct user groups (customers, hotel managers, administrators) whose feature priorities will keep evolving (dynamic pricing, new payment options, review moderation), which fits Agile's iterative, feedback-driven delivery far better than a fixed-scope Waterfall plan.
- Sprint-based delivery lets the team release a minimum viable product — hotel search and booking — early, then add payments, reviews, and admin reporting in subsequent sprints based on real user feedback, reducing the risk of building the wrong features.
- Continuous stakeholder involvement (hotel managers testing pricing tools, customers trialling booking flows each sprint) catches usability and requirement issues early, which is far costlier to fix late in a sequential Waterfall approach.

6. Conclusion

By modelling Users, Bookings, Rooms, and Payments as encapsulated, polymorphic, inheritance-based classes and delivering the platform through Agile sprints, the hotel reservation system can evolve safely alongside changing customer and business needs.

Question 2

University Management System

1. System Understanding, Actors, and Use Cases

The system automates admissions, course registration, examination management, and result processing for students, faculty, examination staff, and administrators.

Actor	Key Use Cases
Student	Apply for Admission, Register Courses, View Timetable, View Results, Pay Fees
Faculty	Manage Course Content, Record Attendance, Enter Marks
Examination Staff	Schedule Exams, Process Results, Issue Grade Reports
Administrator	Manage Users, Manage Departments, Generate Reports

2. Application of Object-Oriented Principles

- **Encapsulation:** StudentRecord hides raw marks and attendance data behind controlled methods such as calculateGPA() and getAttendancePercentage(), so results can only be derived through validated logic, not direct field access.
- **Inheritance:** An abstract Person class holds shared attributes (personId, name, email) and login(); Student, Faculty, ExamStaff, and Administrator extend it, each adding role-specific behaviour like registerCourse() or scheduleExam().
- **Polymorphism:** A generateReport() method behaves differently when invoked on an AdmissionReport, ResultReport, or AttendanceReport object — all subclasses of an abstract Report — letting the reporting subsystem treat every report type uniformly.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
Person (abstract)	personId, name, email	login()
Student	rollNo, program, cgpa	registerCourse(), viewResult()
Faculty	facultyId, department	assignGrades(), manageCourseContent()
ExamStaff	staffId	scheduleExam(), processResults()
Administrator	adminId	manageDepartments(), manageUsers()
Course	courseId, title, credits	addSyllabus()
Enrollment	enrollmentId, semester, status	enroll(), drop()
Examination	examId, date, courseId	scheduleExam()
Result	resultId, marks, grade	calculateGrade()
Department	deptId, name, headOfDept	addCourse()

4. Class Relationships

- Generalization: Person is the superclass of Student, Faculty, ExamStaff, and Administrator.
- Association class: Student and Course are linked many-to-many through Enrollment, which carries semester and status data that belongs to neither class alone.
- Aggregation: Department aggregates Course objects — courses can be reassigned between departments without being destroyed.
- Association: Faculty teaches Course; Examination produces Result (1-to-many).

5. SDLC Model Recommendation: Incremental Model

- The system decomposes naturally into well-understood, largely independent modules — admissions, course registration, examination management, and result processing — each of which can be specified, built, and deployed as a separate increment.
- Delivering the admissions module first lets the university start using the system for an upcoming intake while registration and examination modules are still being developed, providing early business value rather than waiting for one big-bang release.
- Because academic processes are reasonably stable and well documented, requirements risk is lower than in consumer-facing systems, so the heavier risk-analysis overhead of the Spiral model is unnecessary — Incremental delivery gives the right balance of structure and flexibility, with each increment independently tested and validated by the relevant user group (students, faculty, exam staff) before the next is built.

6. Conclusion

Modelling the university's operations around a Person hierarchy, an Enrollment association class, and Department aggregation gives a clean, extensible class structure, while incremental delivery lets each module reach students and staff as soon as it is ready, reducing overall project risk.

Question 3

Online Retail Shopping Platform

1. System Understanding, Actors, and Use Cases

The e-commerce platform lets customers browse products, place orders, track deliveries, and pay online, while vendors manage catalogs and administrators monitor transactions and inventory.

Actor	Key Use Cases
Customer	Browse Products, Add to Cart, Place Order, Track Delivery, Make Payment, Write Review
Vendor	Manage Product Catalog, View Orders, Update Stock
Administrator	Monitor Transactions, Manage Inventory, Generate Reports
Delivery Partner	Update Delivery Status

2. Application of Object-Oriented Principles

- **Encapsulation:** Cart and Order hide their internal item lists and totals behind methods such as `addItem()`, `removeItem()`, and `calculateTotal()`, so totals can never be left inconsistent with the item list.
- **Inheritance:** An abstract Product class holds shared attributes (`sku`, `name`, `price`) while `ElectronicsProduct`, `ApparelProduct`, and `GroceryProduct` extend it with category-specific behaviour (e.g., warranty tracking, expiry handling), avoiding duplicated catalog logic.
- **Polymorphism:** `OrderProcessor` calls `product.calculateShippingCost()` polymorphically across any Product subtype, so new product categories can be onboarded without modifying order-processing code.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
User (abstract)	<code>userId</code> , <code>name</code> , <code>email</code>	<code>login()</code>
Customer	<code>shippingAddress</code> , <code>walletBalance</code>	<code>browseProducts()</code> , <code>placeOrder()</code>
Vendor	<code>vendorId</code> , <code>storeName</code>	<code>addProduct()</code> , <code>updateStock()</code>
Administrator	<code>accessLevel</code>	<code>monitorTransactions()</code> , <code>manageInventory()</code>
Product (abstract)	<code>sku</code> , <code>name</code> , <code>price</code> , <code>category</code>	<code>getDetails()</code>
Cart	<code>cartId</code> , <code>items[]</code>	<code>addItem()</code> , <code>removeItem()</code> , <code>calculateTotal()</code>
Order	<code>orderId</code> , <code>orderDate</code> , <code>status</code>	<code>placeOrder()</code> , <code>cancelOrder()</code>
Payment	<code>paymentId</code> , <code>amount</code> , <code>method</code> , <code>status</code>	<code>processPayment()</code>
Delivery	<code>deliveryId</code> , <code>trackingStatus</code> , <code>eta</code>	<code>updateStatus()</code> , <code>trackDelivery()</code>

4. Class Relationships

- Generalization: User is the superclass of Customer, Vendor, and Administrator; Product is the superclass of category-specific product classes.
- Composition: Cart is composed of OrderItem entries; Order is composed of the items purchased at checkout.
- Association: Customer places Order (1-to-many); Order has one Payment and one Delivery; Vendor manages many Product objects.

Achieving Modular Software Design through Object-Oriented Techniques

Modularity is achieved by encapsulating each business capability — catalog, cart, ordering, payment, and delivery

— as an independent, loosely coupled class cluster that communicates only through well-defined interfaces (e.g., an IPaymentGateway interface hides whether payments are processed via card, UPI, or wallet). Abstraction lets the order-processing logic depend on interfaces rather than concrete implementations, so a module such as the payment gateway can be replaced or extended without touching the ordering or catalog modules. Inheritance within the Product hierarchy lets category-specific modules reuse a common base while adding specialised behaviour, and polymorphism allows the order and shipping logic to treat every product uniformly. Together these principles mirror the Open-Closed and Interface-Segregation ideas from SOLID design, producing a platform where individual modules can be developed, tested, deployed, and scaled independently.

5. SDLC Model Recommendation: Agile (Scrum)

- Waterfall is a poor fit because e-commerce requirements — promotions, payment options, UX changes — shift frequently in response to market competition, and Waterfall's rigid, sequential phases make late changes expensive.
- The Spiral model's heavy risk-analysis overhead is more suited to high-risk, novel systems; a retail platform built on well-understood e-commerce patterns does not carry enough technical risk to justify that overhead at every cycle.
- The Incremental model would deliver modules in large blocks but with less frequent stakeholder feedback than the business needs to stay competitive.
- Agile (Scrum) is selected because it delivers working increments every sprint, allowing the team to prioritise high-value features (checkout, catalog search) first, gather continuous feedback from customers and vendors, and adapt quickly to new payment methods or promotional requirements — essential in a fast-moving retail market.

6. Conclusion

A cleanly modularised, interface-driven object model combined with Agile delivery lets the retail platform evolve its catalog, checkout, and delivery features independently and rapidly in response to market and vendor feedback.

Question 4

Smart Waste Management System

1. System Understanding, Actors, and Use Cases

IoT-enabled bins monitor waste levels and automatically notify collection vehicles when full, while citizens can report waste-related issues through a mobile application.

Actor	Key Use Cases
Citizen	Report Waste Issue, View Collection Schedule
Smart Bin (IoT device)	Monitor Waste Level, Trigger Notification (automated)
Collection Vehicle / Driver	Receive Collection Alerts, Update Collection Status
Municipal Administrator	Monitor Bin Status, Manage Collection Routes, Generate Reports

2. Application of Object-Oriented Principles

- **Encapsulation (applied to system design):** The SmartBin class hides raw sensor readings and threshold-comparison logic behind checkLevel() and sendAlert(), so the collection-scheduling subsystem never touches sensor data directly.
- **Inheritance:** An abstract Device class defines shared attributes (deviceId, status, lastPing()); SmartBin and other future IoT device types extend it, reusing common connectivity and health-check behaviour.
- **Polymorphism:** NotificationService.notify() is implemented differently by SMSNotification, AppNotification, and EmailNotification subclasses, so the bin-alert logic can trigger any channel through the same call without knowing the delivery mechanism.
- **Abstraction:** An ICollectionScheduler interface abstracts the route-optimisation algorithm away from vehicle-allocation logic, so the scheduling strategy can be swapped or upgraded independently.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
Device (abstract)	deviceId, status, lastPingTime	lastPing(), getStatus()
SmartBin	binId, location, fillLevel, capacity	checkLevel(), sendAlert()
Sensor	sensorId, readingType, lastReading	takeReading()
Citizen	citizenId, name, contact	reportIssue()
IssueReport	reportId, description, status, timestamp	submitReport(), updateStatus()
CollectionVehicle	vehicleId, driverName, capacity	receiveAlert(), updateStatus()
Route	routeId, binList, estimatedTime	optimizeRoute()
NotificationService (abstract)	notificationId, message	notify()

4. Class Relationships

- Composition: SmartBin is composed of one or more Sensor objects — a sensor has no independent purpose outside its bin.
- Generalization: Device is the superclass of SmartBin (and future IoT device types); NotificationService is the superclass of SMSNotification, AppNotification, and EmailNotification.
- Association/Dependency: SmartBin triggers NotificationService when a threshold is crossed; CollectionVehicle is assigned to a Route; Citizen submits an IssueReport.

5. SDLC Model Recommendation: Spiral Model

- The system combines hardware (sensors, connectivity) with software (real-time alerts, route optimisation, a citizen-facing app), which carries significant technical risk — sensor reliability, network latency, and integration failures — that must be evaluated before committing to full-scale deployment across the municipality.
- Requirements are also expected to evolve as real sensor data patterns and citizen feedback emerge (e.g., adjusting fill-level thresholds or adding new issue categories), which the Spiral model accommodates through repeated planning, risk-analysis, engineering, and evaluation cycles.
- A pilot spiral cycle can prototype sensor integration and notification flow on a small number of bins, allowing the team to identify and mitigate costly hardware or connectivity risks early, before the municipality invests in city-wide rollout — something a purely sequential Waterfall approach cannot support, and something plain Agile does not formally build in as an explicit risk-analysis step.

6. Conclusion

Modelling bins and sensors through composition and IoT-device inheritance, with polymorphic notification handling, gives a flexible object design, while the Spiral model's risk-driven iterations make it the right life-cycle choice for a system that mixes hardware integration risk with evolving citizen-facing requirements.

Question 5

Smart Healthcare Management System

1. System Understanding, Actors, and Use Cases

Patients can book appointments, consult doctors, access medical records, and pay online; doctors update diagnoses and prescriptions; administrators manage departments, staff, and billing.

Actor	Key Use Cases
Patient	Book Appointment, View Medical Records, Make Payment
Doctor	Update Diagnosis, Write Prescription, View Patient History
Hospital Administrator	Manage Departments, Manage Staff, Manage Billing
Payment Gateway (external)	Process Payment

2. Application of Object-Oriented Principles

- **Encapsulation:** MedicalRecord hides sensitive diagnosis and prescription data behind role-checked accessors such as getRecord(requestingUser), ensuring only authorised doctors or the owning patient can view protected health data.
- **Inheritance:** An abstract Person class defines shared attributes (personId, name, contact) and login(); Patient, Doctor, and Administrator extend it, each adding role-specific behaviour such as bookAppointment() or updateDiagnosis().
- **Polymorphism:** A generateBill() method behaves differently for ConsultationBill, PharmacyBill, and LabTestBill — all subclasses of an abstract Bill — so BillingService can total a patient's charges uniformly regardless of bill type.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
Person (abstract)	personId, name, contact	login()
Patient	patientId, dateOfBirth, insuranceInfo	bookAppointment(), viewRecords()
Doctor	doctorId, specialization, department	updateDiagnosis(), writePrescription()
Administrator	adminId	manageStaff(), manageDepartments()
Appointment	appointmentId, date, time, status	scheduleAppointment(), cancelAppointment()
MedicalRecord	recordId, diagnosis, history	updateRecord(), getRecord()
Prescription	prescriptionId, medication, dosage	issuePrescription()
Department	deptId, name, headDoctor	assignDoctor()
Bill (abstract)	billId, amount, status	generateBill(), processPayment()

4. Class Relationships

- Generalization: Person is the superclass of Patient, Doctor, and Administrator; Bill is the superclass of ConsultationBill, PharmacyBill, and LabTestBill.
- Association class: Patient and Doctor are linked through Appointment, which carries date, time, and status data belonging to the relationship itself.
- Composition: MedicalRecord belongs exclusively to one Patient and has no meaning independent of that patient.
- Aggregation: Department aggregates Doctor objects — a doctor can move between departments without being destroyed.

5. SDLC Model Recommendation: Incremental Model

- Healthcare software is safety- and compliance-critical, so each capability — appointment booking, medical records, prescriptions, billing — needs thorough, independently verifiable testing before being relied upon in a clinical setting; the Incremental model allows exactly this, with each increment fully tested and validated before the next is added.
- Delivering the lower-risk appointment-booking module first provides early operational value while the team invests more design and testing effort in higher-risk increments such as medical records and prescriptions, where data accuracy and patient privacy are paramount.
- Compared to Waterfall, which would defer all testing to the end and risk discovering compliance or safety issues too late, and compared to unstructured Agile, which may lack the formal verification gates hospitals require, Incremental development gives the right balance of phased, validated delivery for a system where errors can directly affect patient safety.

6. Conclusion

A Person-based inheritance hierarchy, an Appointment association class, and composition-protected medical records give the healthcare system a safe, well-structured object model, while incremental delivery ensures each clinically sensitive module is rigorously validated before hospital staff and patients depend on it.

Question 6

Smart Library Management System

1. System Understanding, Actors, and Use Cases

Students and faculty can search, borrow, and return books, reserve titles, and pay overdue fines; librarians manage inventories, memberships, and circulation, while administrators generate reports and maintain the system.

Actor	Key Use Cases
Member (Student / Faculty)	Search Books, Borrow Book, Return Book, Reserve Book, Pay Fine
Librarian	Manage Book Inventory, Manage Memberships, Manage Circulation Records
Administrator	Generate Reports, Maintain System

2. Application of Object-Oriented Principles

- **Encapsulation:** Book hides its available-copy count behind borrowCopy() and returnCopy() methods, so the count can never be decremented below zero or left out of sync with actual loans.
- **Inheritance:** An abstract Person class defines shared attributes (personId, name, email); Member (abstract) extends it and is further specialised by Student and Faculty, while Staff (abstract) is specialised by Librarian and Administrator — a two-level hierarchy that reuses common behaviour while letting each concrete role add its own.
- **Polymorphism:** calculateFine() is overridden differently by Student and Faculty subclasses of Member (different grace periods and daily rates), so Loan.processReturn() can call it polymorphically without needing to know the borrower's exact type.

3. Key Classes, Attributes, and Methods

Class	Attributes	Methods
Person (abstract)	personId, name, email	login()
Member (abstract)	memberId, membershipStatus	borrowBook(), reserveBook(), payFine()
Student / Faculty	program / department	calculateFine()
Librarian	staffId	manageInventory(), manageMembership()
Book	isbn, title, author, category	search()
BookCopy	copyId, status	checkAvailability()
Loan	loanId, issueDate, dueDate, returnDate	issueLoan(), returnLoan()
Reservation	reservationId, date, status	reserveBook(), cancelReservation()
Fine	fineId, amount, status	calculateFine(), payFine()

4. Class Relationships

- Generalization: Person is the superclass of Member and Staff; Member is further specialised by Student and Faculty; Staff is further specialised by Librarian and Administrator.
- Composition: Book is composed of individual BookCopy objects — a copy has no meaning without its parent Book title.
- Association class: Member and BookCopy are linked through Loan, which records issue/due/return dates belonging to the borrowing event itself.
- Association: Member reserves Book through Reservation; Loan may generate a Fine (0-or-1 relationship).

5. SDLC Model Recommendation: Waterfall Model

- Library circulation processes — searching, borrowing, returning, reserving, and fining — are a well-established, stable domain with clearly defined rules that are unlikely to change significantly during development, unlike consumer-facing or IoT-driven systems.
- With low requirements uncertainty and a moderate, well-scoped feature set, the sequential Waterfall phases (requirements, design, implementation, testing, deployment) allow thorough upfront analysis and complete documentation, which suits an institutional system that must be handed over to library staff with clear, stable specifications.
- The heavier iterative overhead of Agile or the risk-analysis cycles of the Spiral model are not justified here, since the technical and requirements risk is low; should the library later want to add features incrementally (e.g., a recommendation engine), the team can adopt an Incremental approach for that extension without having needed it for the core system.

6. Conclusion

A two-level Person-Member/Staff inheritance hierarchy, Book-BookCopy composition, and a Loan association class produce a clean, well-structured model for library operations, and the Waterfall model's structured, sequential phases fit this stable, well-understood domain well.